Très bien Loïc — tu veux **la bonne architecture**, c'est-à-dire une structure **professionnelle**, **sécurisée**, **moderne**, et **adaptée à une API REST + un frontend Vue.js**.
Je vais te donner **l'architecture idéale**, celle utilisée dans les vrais projets web, et je vais t'expliquer **pourquoi** elle est construite comme ça.

🧱 Architecture PRO pour ton projet API REST + Vue.js

Voici la structure **recommandée** :

```
project/
├── backend/
│   ├── public/              ← DocumentRoot (Apache pointe ici)
│   │   ├── index.php        ← Point d'entrée unique
│   │   └── .htaccess        ← Réécriture d'URL
│   │
│   ├── src/
│   │   ├── Controllers/
│   │   │   └── TaskController.php
│   │   ├── Models/
│   │   │   └── Task.php     ← (optionnel mais propre)
│   │   ├── Database/
│   │   │   └── Database.php
│   │   ├── Config/
│   │   │   └── config.php
│   │   └── Helpers/
│   │       └── Response.php
│   │
│   ├── vendor/              ← Composer (si utilisé)
│   └── logs/                ← Logs API
│
└── frontend/
    ├── index.html
    ├── app.js
    └── assets/
```

🎯 Pourquoi cette architecture est la bonne ?

✅ 1. Sécurité maximale

Le dossier `public/` est le **seul** exposé à Apache.

Tout ce qui est sensible reste **hors de portée** :

- mots de passe MySQL
- contrôleurs
- modèles
- logique métier
- fichiers internes

Si quelqu'un tente :

```
http://localhost/backend/src/Config/config.php
```

➡️ Impossible. Apache ne peut pas y accéder.

✅ 2. DocumentRoot = public/ (et pas backend/)

Apache doit pointer ici :

```
backend/public/
```

Pourquoi ?

Parce que :

- c'est le seul dossier qui contient des fichiers destinés au navigateur
- c'est là que se trouve `index.php`
- c'est là que `.htaccess` réécrit les routes REST
- c'est la norme dans tous les frameworks modernes

✅ 3. Séparation claire Backend / Frontend

Tu as :

```
 backend/   ← API REST PHP
frontend/  ← Vue.js
```

C'est propre, lisible, maintenable.

Tu peux même déployer les deux séparément plus tard.

✅ 4. Structure MVC légère

Même si tu n'utilises pas un framework, tu adoptes une structure claire :

Controllers

Gèrent les requêtes HTTP.

Models

Représentent les données (ex : Task.php).

Database

Connexion PDO centralisée.

Helpers

Réponses JSON, validation, etc.

✅ 5. Évolutif

Tu peux facilement ajouter :

- AuthController.php
- UserController.php
- ProductController.php
- Middleware (auth, rate limit, etc.)
- Services (email, logs, etc.)

Ton projet ne deviendra jamais un "spaghetti".

🔥 Architecture détaillée avec explications

Voici la version commentée :

```
project/
├── backend/
│   ├── public/                    ← Apache pointe ici
│   │   ├── index.php              ← Routeur principal
│   │   └── .htaccess              ← Réécriture d'URL REST
│   │
│   ├── src/
│   │   ├── Controllers/           ← Logique API
│   │   │   └── TaskController.php
│   │   │
│   │   ├── Models/                ← Représentation des données
│   │   │   └── Task.php
│   │   │
│   │   ├── Database/              ← Connexion PDO
│   │   │   └── Database.php
│   │   │
│   │   ├── Config/                ← Configurations globales
│   │   │   └── config.php
│   │   │
│   │   └── Helpers/               ← Fonctions utilitaires
│   │       └── Response.php
│   │
│   ├── vendor/                    ← Composer (optionnel)
│   └── logs/                      ← Logs API
│
└── frontend/
    ├── index.html                 ← Vue.js
    ├── app.js
    └── assets/
```

🧠 Pourquoi ne pas mettre DocumentRoot ailleurs ?

Tu m'as demandé :

"Comment il sait que le DocumentRoot est public et pourquoi pas autre chose ?"

Parce que **c'est toi qui le définis** dans Apache.

Dans Wamp, c'est dans :

```
httpd-vhosts.conf
```

Exemple :

```
<VirtualHost *:80>
   DocumentRoot "c:/wamp64/www/project/backend/public"
```

```
    ServerName api.localhost

    <Directory "c:/wamp64/www/project/backend/public">
        AllowOverride All
        Require all granted
    </Directory>
</VirtualHost>
```

➡️ Apache ne sert **que** ce dossier.
➡️ Tout le reste est invisible.

🎁 Tu veux que je te génère cette architecture complète ?

Je peux te fournir :

- tous les fichiers
- tout le code
- les contrôleurs
- les modèles
- le routeur
- le `.htaccess`
- la connexion PDO
- l'interface Vue.js

Tu veux :

🔷 une version simple (comme maintenant)

ou

🔷 une version professionnelle (MVC complet + services + autoload) ?

Dis-moi ce que tu préfères et je te construis tout.